W3. Code Profiling

Department of Smart Mobility Engineering
Inwook, Shim
인하대학교
INHA UNIVERSITY

Today

- Goal
  - Any measurable resource can be profiled (CPU, memory, network…)
  - In this class, we will focus on CPU time and memory usage.
  - Get rid of your believe-ness on your code.
    - Rather than your intuition, it is more sensible to profiling first.
    - **Display and See Data**
- Content
  - Julia Set
  - Timing
  - Memory
  - Using modules

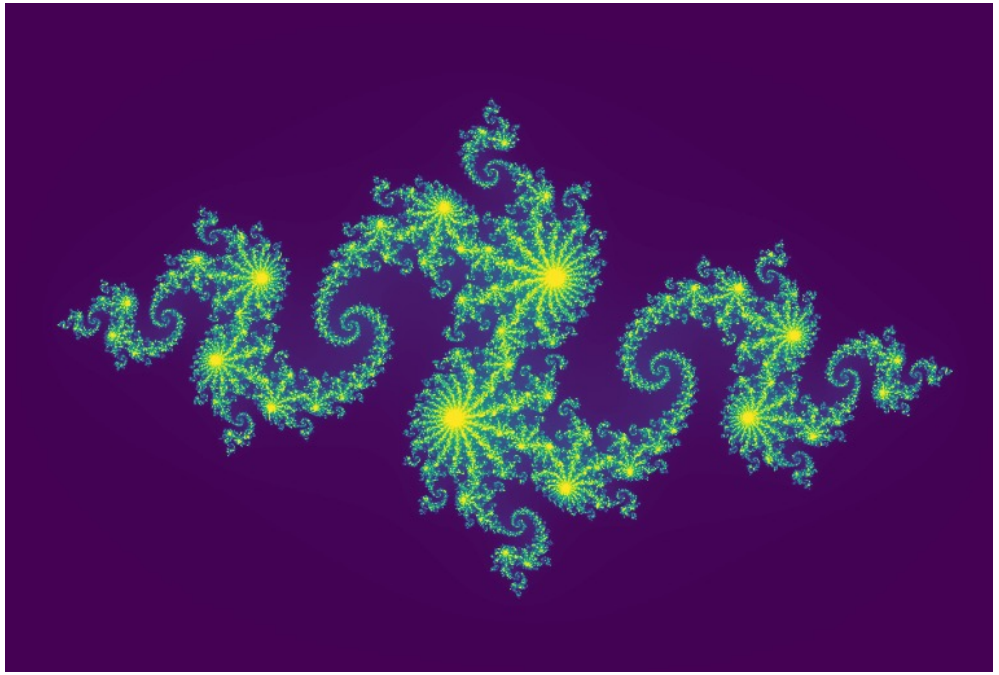
Efficient Profiling

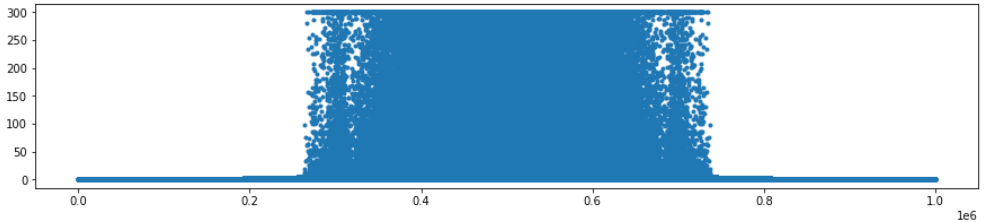
Julia set

- Fractal sequence generates a complex output image.
- Each pixel is <u>isolated</u>, that is no data sharing between pixels.

$f(z) = z^2 + c$, where z and c are complex number

```
#PSEUDO-CODE
for z in range(coordinates):
  while abs(z) < 2 and n < maxiter:
    z = z * z + c
    n += 1
  output[coordinate] = z
return output
```



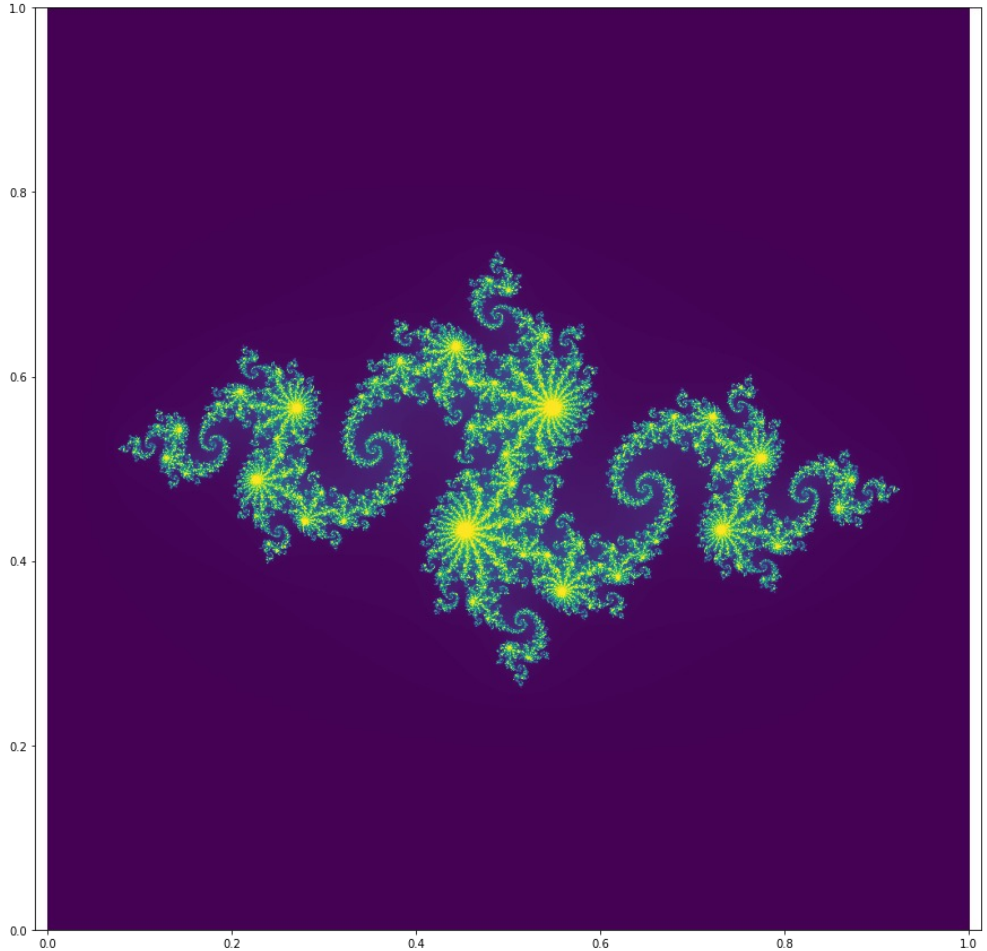


https://en.wikipedia.org/wiki/Julia_set

Julia set calculation example

```
def calculate_Julia_set (maxiter, zs, cs):
  """Calculate output list using Julia update rule"""
  output = [0] * len(zs)
  for i in range(len(zs)):
    n = 0
    z = zs[i]
    c = cs[i]
    while abs(z) < 2 and n < maxiter:
      z = z * z + c
      n += 1
    #end while
    output[i] = n
  #end for
  return output
```

All memories are prepared before calculation.



Julia set calculation example

```
def build_julia_set(desired_width, max_iterations):
  """Create a list of complex coordinates (zs) and complex parameters (cs), build Julia set"""
  x_step = (x2 - x1) / desired_width
  y_step = (y1 - y2) / desired_width
  x = []
  y = []
  ycoord = y2
  while ycoord > y1:
    y.append(ycoord)
    ycoord += y_step
  xcoord = x1
  while xcoord < x2:
    x.append(xcoord)
    xcoord += x_step
  zs = []
  cs = []
  for ycoord in y:
    for xcoord in x:
      zs.append(complex(xcoord, ycoord))
      cs.append(complex(c_real, c_imag))

  output = calucate_julia_set(max_iterations, zs, cs)
  return output
```

Approaches to Timing 1. print

- The simplest way : Time package & print function

Directly write time checker code into your code.

```
print("Length of x:", len(x))
print("Total elements:", len(zs))
start_time = time.time()
output = calucate_julia_set(max_iterations, zs, cs)
end_time = time.time()
secs = end_time - start_time
print(calucate_julia_set.__name__ + " took", secs, "seconds")
```

Run for testing

```
output = build_julia_set(desired_width=1000, max_iterations=300)
```

```
Length of x: 1000
Total elements: 1000000
calucate_julia_set took 9.661095142364502 seconds
```

- Measurement only once → This may give wrong information to us. (randomness?)

Approaches to Timing 1. print

- 문제점은 있고, 세련되지 않아도 기본이며, 코드를 처음 간단하게 살펴볼 때 매우 유용!
- 디버깅할 때와 마찬가지로 프로파일링에서도 기본이며, 코드가 좀 더러워지지만, 뭐 잘 정리하는 수 밖에 없다!

- 코드의 실행시간을 반복 측정하면 어떠한 패턴이나 정규 분포가 나와야 한다.
- 대상 코드가 무작위로 실행시간이 달라진다면, 다른 프로세스에 의해서 심각하게 성능이 손상되고 있다는 의미!
  >> 이런 경우, 개선할 필요가 없는 코드를 개선 할 수도 있고, 잘못된 개선을 하는 것일 수도 있으므로 주의!

Approaches to Timing 2. decorator

```python
from functools import wraps
def timefn(fn):
  @wraps(fn)
  def measure_time(*args, **kwargs):
    t1 = time.time()
    result = fn(*args, **kwargs)
    t2 = time.time()
    print(f"@timefn: {fn.__name__} took {t2 - t1} seconds")
    return result
  return measure_time
```

*args 가변길이 인자
**kwargs 가변길이 키워드 인자.
@wraps(fn) 데커레이터로 넘어온 함수이름과 독스트링 (docstring) 을 호출 가능하도록 해줌.

```python
@timefn
def calucate_julia_set(maxiter, zs, cs):
  """Calculate output list using Julia update rule"""
  output = [0] * len(zs)
  for i in range(len(zs)):
    n = 0
    z = zs[i]
    c = cs[i]
    while abs(z) < 2 and n < maxiter:
      z = z * z + c
      n += 1
    #end while
    output[i] = n
  #end for
  return output
```

Add decorator before the definition of function.

- 코드를 깔끔하게 관리하기 용이
- 측정방법에 따라서 조금씩 측정 결과가 다르다.
- 별도의 overhead와 real-time os가 아님에 유의하라.

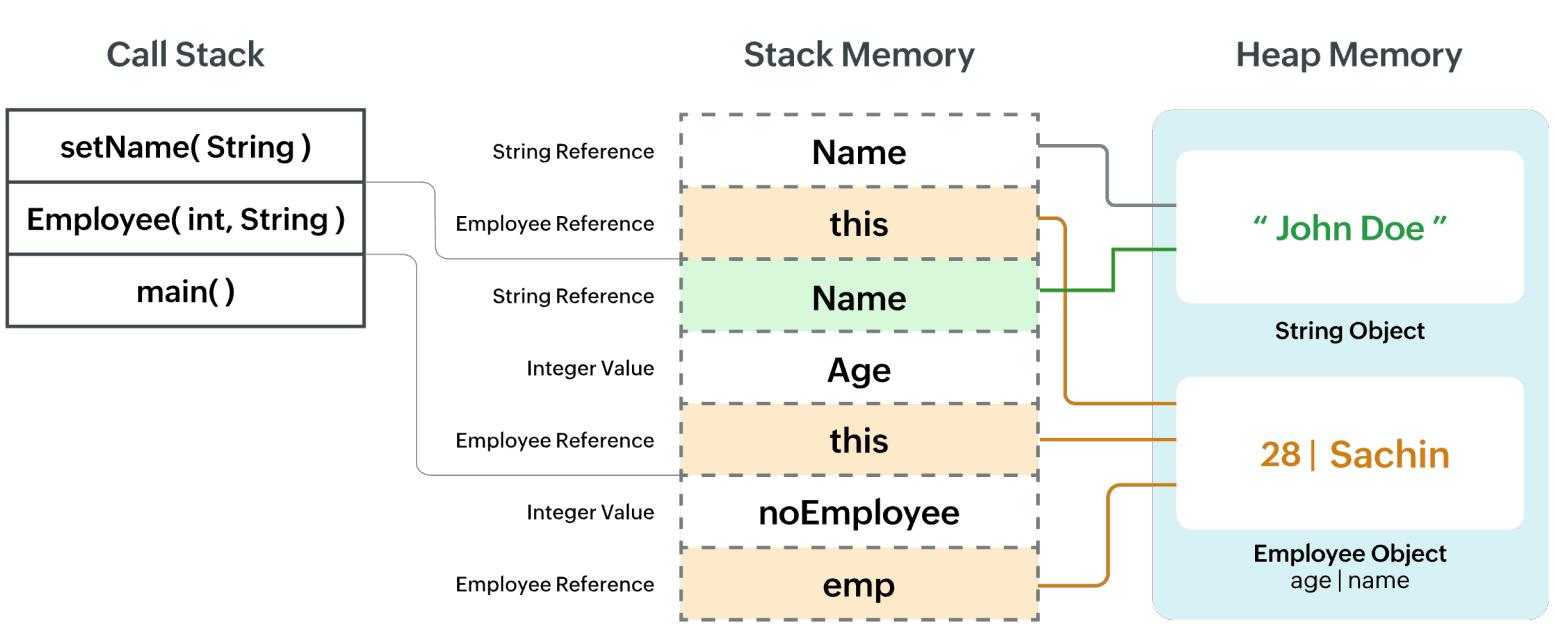
Note

- The addition of profiling will increase the processing time of your code.
- Some profiling options are too informative and induce heavy performance penalty.
- The tread-off btw. profiling detail and code performance should be considered.

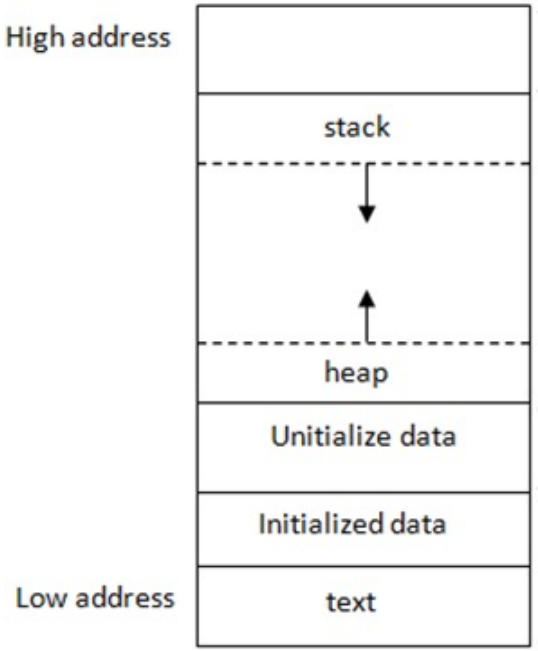
Approaches to Timing 3. `timeit` module

- Get a coarse measurement of the execution speed of CPU time
- `timeit` module temporally disables <u>the garbage collector.</u>

  → This might be making time difference you'll see with real-world operations.



https://www.site24x7.com/learn/java/heap-and-stack-memory-management.html



https://courses.engr.illinois.edu/cs225/sp2022/resources/stack-heap/

Approaches to Timing 3. `timeit` module

-s : build_julia_set을 모듈안에서 정의하였으므로 그 모듈을 임포트해야함
코드를 보면 다른 프로파일링 도구에서 안전하게 임포트할 수 있도록 __main__ 블록으로 쌓아놨음.

```
$ python3 -m timeit -n 5 -r 1 -s "import julia1" "julia1.build_julia_set(
desired_width=1000, max_iterations=300)"
```

-n : the number of loop
-r : the number of repetitions to repeat the experiments

```
root@bfe578b7fa45:/workspace/hpp/w3# python3 -m timeit -n 5 -r 1 -s "import julia1" "julia1.build_julia_set(desired_width=1000, max_iterations=300)"
5 loops, best of 1: 3.22 sec per loop
root@bfe578b7fa45:/workspace/hpp/w3# python3 -m timeit -n 5 -r 2 -s "import julia1" "julia1.build_julia_set(desired_width=1000, max_iterations=300)"
5 loops, best of 2: 3.18 sec per loop
root@bfe578b7fa45:/workspace/hpp/w3# python3 -m timeit -n 5 -r 10 -s "import julia1" "julia1.build_julia_set(desired_width=1000, max_iterations=300)"
5 loops, best of 10: 3.21 sec per loop
```

- In real-world, try running the time profiling several times to check if you get varying results. (There is no rule to set configuration.)
- Do more repetitions until your profiling result is stable.

Approaches to Timing 3. `timeit` module

- In jupyter nootbook (e.g., Colab), use the magic command `%timeit` as follows:

```
%timeit output = build_julia_set(desired_width=1000, max_iterations=300)
```

```
>> 6.67 s ± 242 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

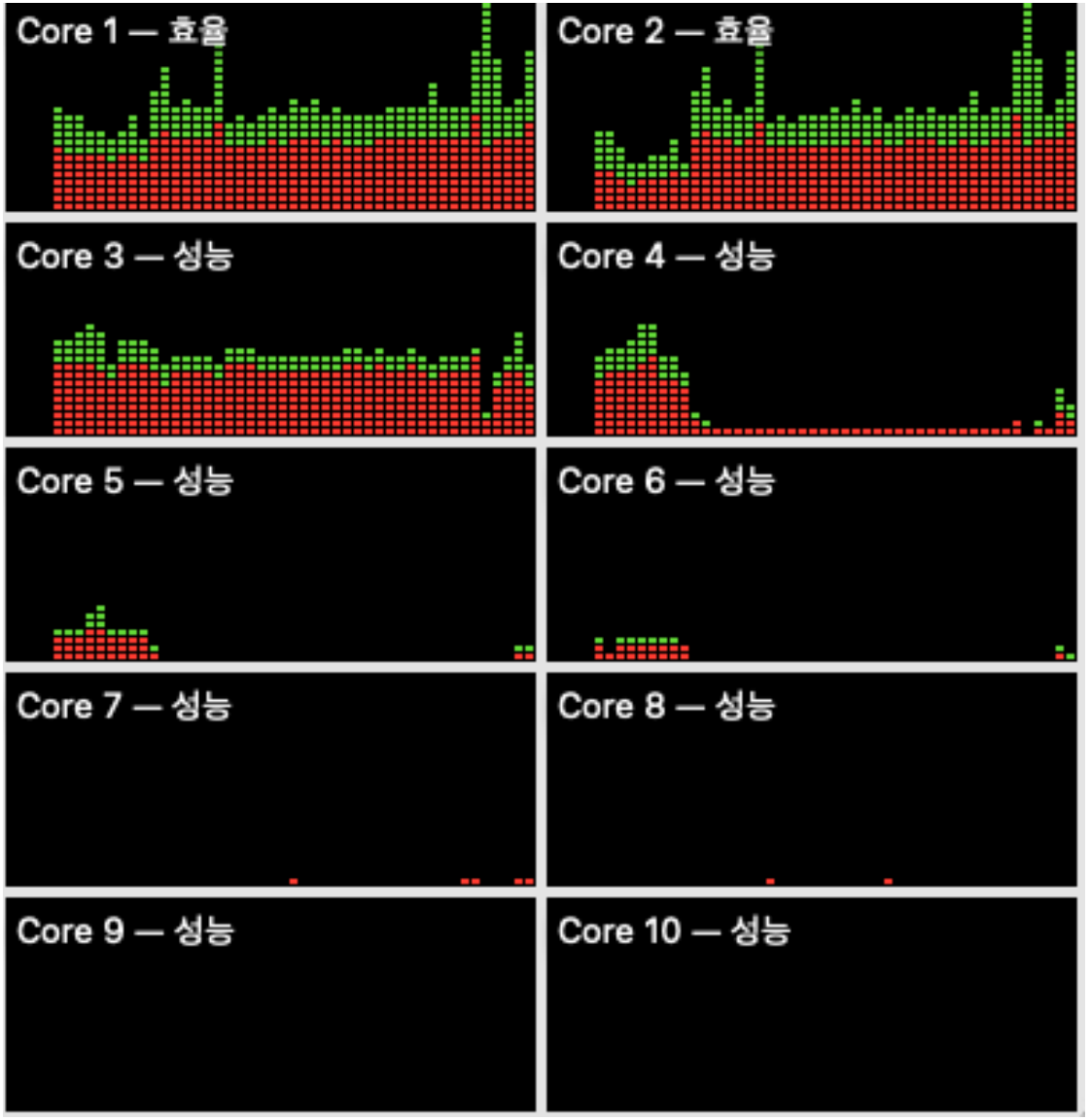
- Question: difference between `timeit.py` and `%timeit`

  Ans; timeit.py 는 best 결과 값을, %timeit 는 평균과 표준편차를 보여주는 방식으로 2016년부터 변경되었음

- <u>Do not use mix them together when you are profiling.</u>

Performance variance

- In normal computer of ours, lots of background tasks (apps) are running that could impact the CPU performance.



- 우리가 사용하는 일반적인 컴퓨터에서 확인할 수 있는 이러한 CPU 편차는 한번 생각해볼만 하다.
- 백그라운드에서 실행 중인 다양한 프로세스는 CPU, RAM, 디스크 자원에 계속해서 접근

Approaches to Timing 4. ubuntu time command

- real: wall clock, total elapse time
- user: the amount of the the CPU spent on the task outside of kerner func.
- sys: the time spent in kernel functions.

```
$ docker run -it -v /home/user/workspace/:/workspace --cpus 1 --name ubuntu_cpu1 ubuntu:latest

root@0e907946a037:/workspace/hpp/w3# time -p python3 julia1.py
real 7.96
user 7.78
sys 0.18

$ docker run -it -v /home/user/workspace/:/workspace --name ubuntu_cpu1 ubuntu:latest

root@01e7361ded93:/workspace/hpp/w3# time -p python3 julia1.py
real 7.87
user 8.50
sys 3.33
```

* It is not limited to Python because `time` is a unix command.

Approaches to Timing 4. ubuntu time command

```
root@bfe578b7fa45:/workspace/hpp/w3# time --verbose python3 julia1.py
        Command being timed: "python3 julia1.py"
        User time (seconds): 3.91
        System time (seconds): 0.05
        Percent of CPU this job got: 109%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:03.62
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 142224
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 31970
        Voluntary context switches: 15
        Involuntary context switches: 134
        Swaps: 0
        File system inputs: 0
        File system outputs: 0
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
```

install time module to your system

```
>> apt install time
```

add following command to the end of the .bashrc

```
alias time='/usr/bin/time'
```

Approaches to Timing 4. ubuntu time command

- Page fault
- 운영체제가 RAM에서 필요한 데이터를 못 찾을 경우 디스크에서 페이지를 얼마나 많이 새로 불러왔는지를 나타냄
- 메모리를 많이 사용하는 프로세스나 프로그램을 한번에 여러가지를 실행할 때, Page fault가 많이 나는 코드가 있다면, 어떤 프로그램이 디스크 접근 때문에 속도가 느려지는지에 대해서 운영체제 수준에서 확인가능

Approaches to Timing 5. cProfile module

```
python3 -m cProfile -s cumulative julia1.py
```

```
root@bfe578b7fa45:/workspace/hpp/w3# python3 -m cProfile -s cumulative julia1.py
         38534991 function calls (38523752 primitive calls) in 9.854 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
    257/1    0.001    0.000    9.855    9.855 {built-in method builtins.exec}
        1    0.019    0.019    9.855    9.855 julia1.py:1(<module>)
        1    0.386    0.386    9.044    9.044 julia1.py:33(build_julia_set)
        1    5.744    5.744    8.525    8.525 julia1.py:20(calculate_julia_set)
 34220076    2.781    0.000    2.781    0.000 {built-in method builtins.abs}
       28    0.001    0.000    0.919    0.033 __init__.py:1(<module>)
    313/1    0.001    0.000    0.791    0.791 <frozen importlib._bootstrap>:1022(_find_and_load)
    310/1    0.001    0.000    0.791    0.791 <frozen importlib._bootstrap>:987(_find_and_load_unlocked)
    295/2    0.001    0.000    0.791    0.396 <frozen importlib._bootstrap>:664(_load_unlocked)
    249/2    0.000    0.000    0.791    0.396 <frozen importlib._bootstrap_external>:877(exec_module)
    442/2    0.000    0.000    0.791    0.395 <frozen importlib._bootstrap>:233(_call_with_frames_removed)
    420/6    0.000    0.000    0.737    0.123 {built-in method builtins.__import__}
   477/54    0.001    0.000    0.702    0.013 <frozen importlib._bootstrap>:1053(_handle_fromlist)
        1    0.000    0.000    0.656    0.656 pyplot.py:1(<module>)
 1056/922    0.010    0.000    0.586    0.001 {built-in method builtins.__build_class__}
      101    0.000    0.000    0.529    0.005 artist.py:129(_update_set_signature_and_docstring)
      100    0.000    0.000    0.525    0.005 artist.py:105(__init_subclass__)
        1    0.000    0.000    0.360    0.360 figure.py:1(<module>)
      114    0.000    0.000    0.345    0.003 artist.py:1743(kwdoc)
```

Approaches to Timing 5. cProfile module

```
python3 -m cProfile -s cumulative julia1.py
python3 -m cProfile -o profile.stats julia1.py
```

```
[root@bfe578b7fa45:/workspace/hpp/w3# python3 -m cProfile -s cumulative julia1.py
         38534991 function calls (38523752 primitive calls) in 9.854 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
    257/1    0.001    0.000    9.855    9.855 {built-in method builtins.exec}
        1    0.019    0.019    9.855    9.855 julia1.py:1(<module>)
        1    0.386    0.386    9.044    9.044 julia1.py:33(build_julia_set)
        1    5.744    5.744    8.525    8.525 julia1.py:20(calculate_julia_set)
 34220076    2.781    0.000    2.781    0.000 {built-in method builtins.abs}
       28    0.001    0.000    0.919    0.033 __init__.py:1(<module>)
    313/1    0.001    0.000    0.791    0.791 <frozen importlib._bootstrap>:1022(_find_and_load)
    310/1    0.001    0.000    0.791    0.791 <frozen importlib._bootstrap>:987(_find_and_load_unlocked)
    295/2    0.001    0.000    0.791    0.396 <frozen importlib._bootstrap>:664(_load_unlocked)
    249/2    0.000    0.000    0.791    0.396 <frozen importlib._bootstrap_external>:877(exec_module)
    442/2    0.000    0.000    0.791    0.395 <frozen importlib._bootstrap>:233(_call_with_frames_removed)
    420/6    0.000    0.000    0.737    0.123 {built-in method builtins.__import__}
   477/54    0.001    0.000    0.702    0.013 <frozen importlib._bootstrap>:1053(_handle_fromlist)
        1    0.000    0.000    0.656    0.656 pyplot.py:1(<module>)
 1056/922    0.010    0.000    0.586    0.001 {built-in method builtins.__build_class__}
      101    0.000    0.000    0.529    0.005 artist.py:129(_update_set_signature_and_docstring)
      100    0.000    0.000    0.525    0.005 artist.py:105(__init_subclass__)
        1    0.000    0.000    0.360    0.360 figure.py:1(<module>)
      114    0.000    0.000    0.345    0.003 artist.py:1743(kwdoc)
```

Max=1000*1000*300=300,000,000

Approaches to Timing 5. cProfile module

```
Python 3.10.4 (main, Jun 29 2022, 12:14:53) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import pstats
>>> p = pstats.Stats("profile.stats")
>>> p.sort_stats("cumulative")
<pstats.Stats object at 0xffff85c508e0>
>>> p.print_stats()
```

DIY

```
>>> p.print_callers()
>>> p.print_callees()
```

```
>>> p.print_stats()
Fri Aug 26 01:11:54 2022    profile.stats

         38535075 function calls (38523836 primitive calls) in 9.890 seconds

   Random listing order was used

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      250    0.000    0.000    0.000    0.000 {method 'get' of 'mappingproxy' objects}
      171    0.000    0.000    0.000    0.000 {method 'values' of 'mappingproxy' objects}
      101    0.000    0.000    0.000    0.000 {method 'items' of 'mappingproxy' objects}
        3    0.000    0.000    0.000    0.000 {method 'copy' of 'mappingproxy' objects}
      156    0.000    0.000    0.000    0.000 {method '__contains__' of 'frozenset' objects}
     1937    0.000    0.000    0.000    0.000 {method 'add' of 'set' objects}
       18    0.000    0.000    0.000    0.000 {method 'discard' of 'set' objects}
        1    0.000    0.000    0.000    0.000 {method 'difference_update' of 'set' objects}
       47    0.000    0.000    0.000    0.000 {method 'issuperset' of 'set' objects}
       35    0.000    0.000    0.000    0.000 {method 'pop' of 'set' objects}
        4    0.000    0.000    0.000    0.000 {method 'remove' of 'set' objects}
        1    0.000    0.000    0.000    0.000 {method 'union' of 'set' objects}
       17    0.000    0.000    0.000    0.000 {method 'update' of 'set' objects}
      780    0.000    0.000    0.000    0.000 {function RcParams.__getitem__ at 0xffff9fda89d0}
    37887    0.003    0.000    0.003    0.000 {method 'get' of 'dict' objects}
     1749    0.000    0.000    0.000    0.000 {method 'setdefault' of 'dict' objects}
      480    0.000    0.000    0.000    0.000 {method 'pop' of 'dict' objects}
       44    0.000    0.000    0.000    0.000 {method 'keys' of 'dict' objects}
      674    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}
       41    0.000    0.000    0.000    0.000 {method 'values' of 'dict' objects}
     1324    0.000    0.000    0.000    0.000 {method 'update' of 'dict' objects}
```

Approaches to Timing 5. `cProfile` module

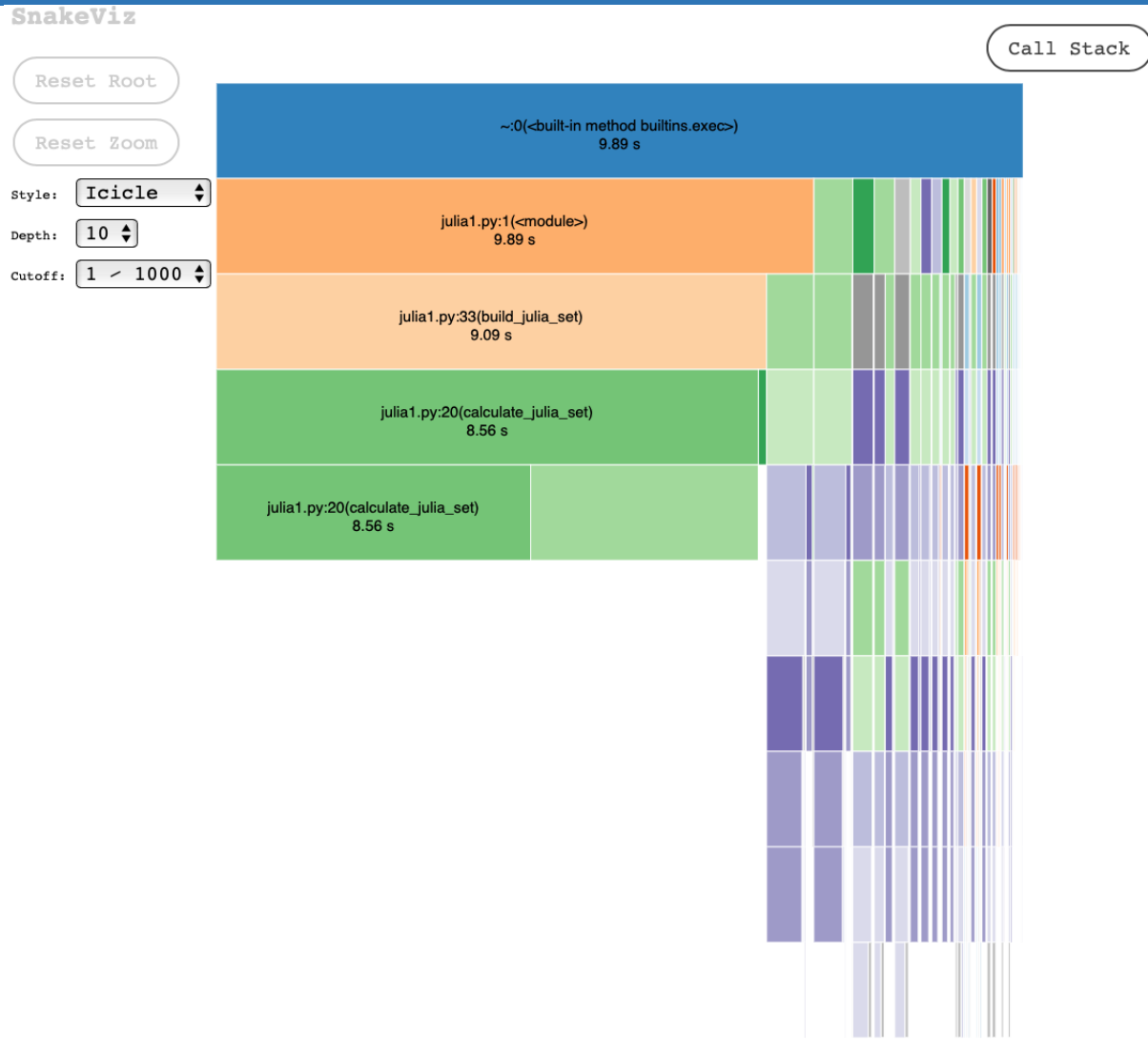
- `cProfile` is rather verbose. (Please carefully watch and read the messages!

  The messages from your computers give lots of clues to find a problem.)
- <u>It is still convenient tool for quickly identifying bottlenet of your code.</u>

Approaches to Timing 6. `sankeviz` module

- `snakeviz` is a visualizing tool that shows the outputs of `cProfile`.
- It is good to understanding your code at the high-level stage.
- When you joint or start a new project, it is a good start point to grasping the whole code.

>>> pip3 install snakeviz



Search:

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	5.835	5.835	8.556	8.556	julia1.py:20(calculate_julia_set)
34220076	2.721	7.952e-08	2.721	7.952e-08	~:0(<built-in method builtins.abs>)
1	0.3999	0.3999	9.09	9.09	julia1.py:33(build_julia_set)

Approaches to Timing 7. `line_profiler` module

>>> pip3 install line_profiler

Add @profile decorator to what you want to profile line-by-line

```
@profile
def calculate_julia_set(maxiter, zs, cs):
  """Calculate output list using Julia update rule"""
  output = [0] * len(zs)
  for i in range(len(zs)):
    n = 0
    z = zs[i]
    c = cs[i]
    while abs(z) < 2 and n < maxiter:
      z = z * z + c
      n += 1
    output[i] = n
  return output
```

Approaches to Timing 7. `line_profiler` module

```
root@docker-desktop:/workspace/hpp/w3# kernprof -l -v julia1.py
Wrote profile results to julia1.py.lprof
Timer unit: 1e-06 s

Total time: 34.4074 s
File: julia1.py
Function: calculate_julia_set at line 20

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    20                                           @profile
    21                                           def calculate_julia_set(maxiter, zs, cs):
    22                                             """Calculate output list using Julia update rule"""
    23         1       3954.0   3954.0      0.0    output = [0] * len(zs)
    24   1000001     288293.0      0.3      0.8    for i in range(len(zs)):
    25   1000000     280080.0      0.3      0.8      n = 0
    26   1000000     319092.0      0.3      0.9      z = zs[i]
    27   1000000     289383.0      0.3      0.8      c = cs[i]
    28  34219980   12785364.0      0.4     37.2      while abs(z) < 2 and n < maxiter:
    29  33219980   10350186.0      0.3     30.1        z = z * z + c
    30  33219980    9772526.0      0.3     28.4        n += 1
    31   1000000     318490.0      0.3      0.9      output[i] = n
    32         1          4.0      4.0      0.0    return output
```

Overhead included

Approaches to Timing 7. `line_profiler` module

```
root@docker-desktop:/workspace/hpp/w3# kernprof -l -v julia1.py
Wrote profile results to julia1.py.lprof
Timer unit: 1e-06 s


Total time: 34.4074 s
File: julia1.py
Function: calculate_julia_set at line 20


Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    20                                           @profile
    21                                           def calculate_julia_set(maxiter, zs, cs):
    22                                             """Calculate output list using Julia update rule"""
    23         1       3954.0   3954.0      0.0    output = [0] * len(zs)
    24   1000001     288293.0      0.3      0.8    for i in range(len(zs)):
    25   1000000     280080.0      0.3      0.8      n = 0
    26   1000000     319092.0      0.3      0.9      z = zs[i]
    27   1000000     289383.0      0.3      0.8      c = cs[i]
    28  34219980   12785364.0      0.4     37.2      while abs(z) < 2 and n < maxiter:
    29  33219980   10350186.0      0.3     30.1        z = z * z + c
    30  33219980    9772526.0      0.3     28.4        n += 1
    31   1000000     318490.0      0.3      0.9      output[i] = n
    32         1          4.0      4.0      0.0    return output
```

Quiz. Which one is fast?

Approaches to Timing 7. `line_profiler` module

```
@profile
def build_julia_set(desired_width, max_iterations):
  """Create a list of complex coordinates (zs) and complex parameters (cs), build Julia set"""
  x_step = (x2 - x1) / desired_width
  y_step = (y1 - y2) / desired_width
  x = []
  y = []
  ycoord = y2
  while ycoord > y1:
    y.append(ycoord)
    ycoord += y_step
  xcoord = x1
  while xcoord < x2:
    x.append(xcoord)
    xcoord += x_step

  zs = []
  cs = []
  for ycoord in y:
    for xcoord in x:
      zs.append(complex(xcoord, ycoord))
      cs.append(complex(c_real, c_imag))

  output = calculate_julia_set(max_iterations, zs, cs)
  return output
```

Approaches to Timing 7. `line_profiler` module

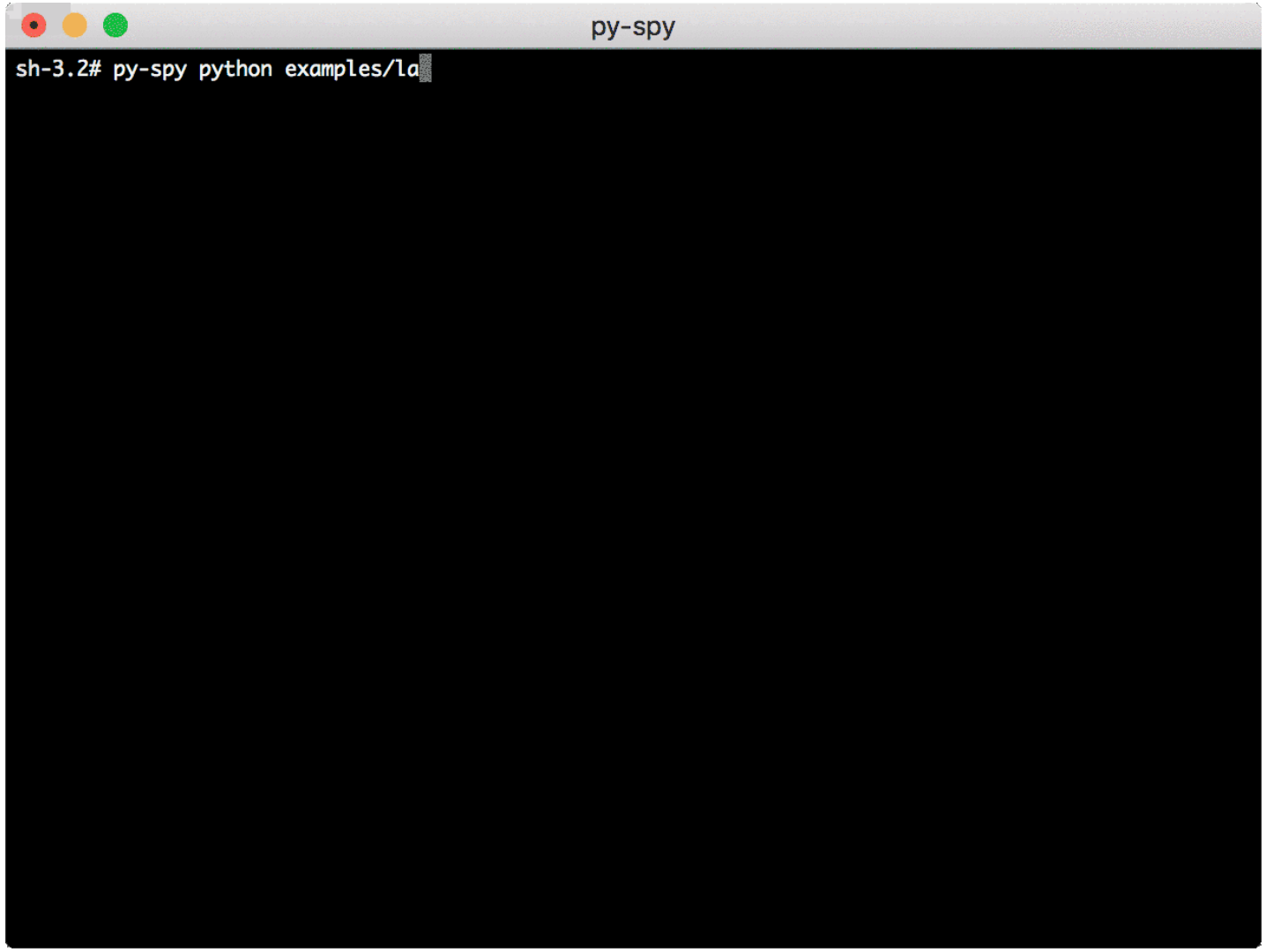
```
root@docker-desktop:/workspace/hpp/w3# kernprof -l -v julia1_nocomment.py
Wrote profile results to julia1_nocomment.py.lprof
Timer unit: 1e-06 s

Total time: 14.2936 s
File: julia1_nocomment.py
Function: build_julia_set at line 33

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    33                                           @profile
    34                                           def build_julia_set(desired_width, max_iterations):
    35                                             """Create a list of complex coordinates (zs) and complex parameters (cs), build Julia set"""
    36         1          2.0      2.0      0.0    x_step = (x2 - x1) / desired_width
    37         1          1.0      1.0      0.0    y_step = (y1 - y2) / desired_width
    38         1          1.0      1.0      0.0    x = []
    39         1          0.0      0.0      0.0    y = []
    40         1          1.0      1.0      0.0    ycoord = y2
    41      1001        318.0      0.3      0.0    while ycoord > y1:
    42      1000        323.0      0.3      0.0      y.append(ycoord)
    43      1000        292.0      0.3      0.0      ycoord += y_step
    44         1          0.0      0.0      0.0    xcoord = x1
    45      1001        283.0      0.3      0.0    while xcoord < x2:
    46      1000        304.0      0.3      0.0      x.append(xcoord)
    47      1000        274.0      0.3      0.0      xcoord += x_step
    48         1          0.0      0.0      0.0    zs = []
    49         1          0.0      0.0      0.0    cs = []
    50      1001        381.0      0.4      0.0    for ycoord in y:
    51   1001000     361028.0      0.4      2.5      for xcoord in x:
    52   1000000     409321.0      0.4      2.9        zs.append(complex(xcoord, ycoord))
    53   1000000     419894.0      0.4      2.9        cs.append(complex(c_real, c_imag))
    54
    55         1   13097025.0 13097025.0     91.6    output = calculate_julia_set(max_iterations, zs, cs)
    56         1       4174.0   4174.0      0.0    assert sum(output) == 33219980
    57         1          1.0      1.0      0.0    return output
```

Memory allocation and initialization

Approaches to Timing 8. py-spy module

- py-spy : Sampling profiler for Python programs, inspecting a currently running process

py-spy top --pid 12345



https://github.com/benfred/py-spy

Conclusion

- Please never avoid profiling based on your intuition.
- Always judge based on measurement results.
- Quick-and-dirty profiling is a good start.
- Our time is more valuable than CPU time.

- <u>In Mobility System</u>
  - CPU Usage and Performance balance (btw. cores/multi-computers/CPU and memory) are directly related to energy consumption and heat problem.
- One more thing
  - Test Driven Development (TDD)